

AdvisorAI: A Retrieval-Augmented Generation System with Fine-Tuned LLaMA for Domain-Specific Academic Advising

Nitin Chaube, Paras Jadhav, Keval Sompura
Stevens Institute of Technology
Hoboken, NJ 07030, USA
{nchaube, pjadhav, ksompura}@stevens.edu

Abstract—Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation, yet they frequently produce hallucinated or outdated information when applied to domain-specific institutional queries. This paper presents AdvisorAI, an intelligent academic advising chatbot designed for Stevens Institute of Technology that combines Retrieval-Augmented Generation (RAG) with cloud-hosted LLMs (Google Gemini and OpenAI) to deliver accurate, context-aware responses about academic programs, courses, faculty, and campus resources. Our system employs a novel multi-agent architecture orchestrated via LangGraph, featuring a ReAct (Reason-Act-Observe) pipeline augmented with a self-reflection quality gate. We introduce a hybrid retrieval mechanism that uses deterministic entity detection with sub-millisecond latency to route queries to targeted ChromaDB collections before falling back to semantic search. In parallel, we conduct a fine-tuning study using QLoRA (Quantized Low-Rank Adaptation) on LLaMA-2-7B with a curated dataset of approximately 87,000 question-answer pairs scraped and generated from official university web pages, targeting future self-hosted generation to reduce API dependency. The system is deployed as a production web application with real-time streaming responses, Firebase authentication, and an administrative dashboard. Our architecture demonstrates that combining entity-aware retrieval augmentation, agentic orchestration, and domain-specific fine-tuning research yields a robust, low-hallucination academic advising system suitable for real-world university deployment.

Index Terms—Retrieval-Augmented Generation, LLaMA, QLoRA, Fine-Tuning, Academic Advising, LangGraph, Multi-Agent Systems, ChromaDB, Hybrid Retrieval

1. Introduction

1.1. Motivation

Academic advising is a critical component of the student experience at universities, influencing course selection, degree planning, career preparation, and overall student satisfaction. However, traditional advising systems face several persistent challenges: limited availability of human advisors, inconsistent information across departments, and

the difficulty of maintaining up-to-date knowledge across rapidly evolving academic programs. Students often resort to searching fragmented institutional websites, consulting outdated catalogs, or relying on peer networks for information that may be inaccurate or incomplete.

The emergence of Large Language Models (LLMs) such as GPT-4, LLaMA, and Gemini has created new possibilities for automated question-answering systems. However, directly applying general-purpose LLMs to institutional advising poses significant risks: these models lack access to current institutional data, are prone to hallucination on domain-specific queries, and cannot be easily updated as programs and policies change.

1.2. Challenges

Building a reliable academic advising chatbot presents several technical challenges:

- 1) **Domain specificity:** The system must accurately represent institutional policies, course offerings, faculty information, and program requirements that are unique to the university and change frequently.
- 2) **Hallucination mitigation:** General-purpose LLMs frequently fabricate plausible-sounding but incorrect information about specific courses, prerequisites, and faculty research areas.
- 3) **Query diversity:** Students ask questions ranging from simple factual lookups (“What are the prerequisites for FE 621?”) to complex multi-hop reasoning (“Which professors in Financial Engineering work on machine learning?”).
- 4) **Real-time responsiveness:** An advising chatbot must provide streaming, low-latency responses to maintain user engagement.
- 5) **Information freshness:** Academic information changes each semester, requiring mechanisms to incorporate new data without full system retraining.

1.3. Contributions

This paper makes the following contributions:

- **AdvisorAI System:** A production-grade academic advising chatbot powered by Google Gemini and

OpenAI APIs, augmented with retrieval-augmented generation for Stevens Institute of Technology.

- **Entity-Aware Hybrid Retrieval:** A novel retrieval mechanism combining deterministic entity detection (course codes, faculty names, intent classification) with semantic search over ChromaDB, achieving sub-millisecond routing latency.
- **LangGraph Multi-Agent Orchestration:** A ReAct + Reflection pipeline implemented in LangGraph that coordinates specialized agents with self-critique quality gates.
- **QLoRA Fine-Tuning Study:** A parallel research effort applying parameter-efficient fine-tuning (QLoRA) to LLaMA-2-7B on a curated dataset of $\sim 87,000$ domain-specific Q&A pairs, designed for future integration as a self-hosted generation backbone.
- **Full-Stack Deployment:** A complete web application with React frontend, FastAPI/Flask backend, Firebase authentication, SSE streaming, and Docker-based cloud deployment on Google Cloud Run.

1.4. Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the system architecture. Section 4 details the data collection and the parallel LLaMA-2-7B fine-tuning effort. Section 5 presents the retrieval-augmented generation pipeline. Section 6 describes the multi-agent orchestration and the production LLM integration. Section 7 covers the full-stack implementation. Section 8 discusses results and evaluation. Section 9 concludes with future directions.

2. Related Work

2.1. Large Language Models

The transformer architecture [1] has given rise to a family of increasingly capable language models. GPT-4 [2] and its predecessors demonstrated strong performance across diverse NLP tasks. Meta’s LLaMA family [3], [4] made high-quality open-weight models accessible for research and fine-tuning. Google’s Gemini [5] introduced multi-modal capabilities with efficient inference. These models form the foundation upon which domain-specific systems like AdvisorAI are built.

2.2. Retrieval-Augmented Generation

RAG [6] addresses LLM hallucination by grounding generation in retrieved documents. The original RAG framework combines a dense retriever with a sequence-to-sequence generator. Subsequent work has explored hybrid retrieval strategies [7], entity-aware retrieval [8], and multi-step retrieval pipelines [9]. Our work extends this line by introducing domain-specific entity detection that operates at sub-millisecond latency, bypassing the overhead of embedding-based routing for structured queries.

2.3. Parameter-Efficient Fine-Tuning

Full fine-tuning of large language models is computationally prohibitive for most organizations. LoRA (Low-Rank Adaptation) [10] addresses this by injecting trainable low-rank decomposition matrices into transformer layers while keeping the base model frozen. QLoRA [11] further reduces memory requirements by quantizing the base model to 4-bit precision during fine-tuning. These methods enable fine-tuning of 7B+ parameter models on consumer-grade GPUs while maintaining competitive performance. Our work applies QLoRA to adapt LLaMA-2-7B for the academic advising domain.

2.4. Agentic AI and Multi-Agent Systems

Recent advances in LLM-based agents [12] have explored systems where language models reason about tool use, plan multi-step actions, and coordinate with other agents. The ReAct framework [13] interleaves reasoning traces with actions, enabling more grounded decision-making. LangGraph [14] provides a graph-based framework for building stateful, multi-agent applications. Self-reflection mechanisms [15] enable LLMs to critique and refine their own outputs. AdvisorAI combines these paradigms in a unified orchestration pipeline.

2.5. Academic Chatbots

Several university chatbot systems have been proposed [16]–[18]. Early systems relied on rule-based approaches or simple retrieval models. More recent work has explored fine-tuned transformer models for educational Q&A [19]. However, few systems combine fine-tuning, RAG, and agentic orchestration in a single production-grade deployment, which is the approach we present here.

3. System Architecture

3.1. Overview

AdvisorAI follows a modular, layered architecture consisting of four primary layers: (1) the presentation layer (React frontend), (2) the API layer (FastAPI + Flask), (3) the intelligence layer (LangGraph orchestrator with multi-agent pipeline), and (4) the data layer (MongoDB, ChromaDB, web scraping). Figure 1 illustrates the high-level architecture.

3.2. Design Principles

The architecture is guided by several key design principles:

- **Modularity:** Each component (retrieval, generation, safety, reflection) operates as an independent module that can be replaced or upgraded without affecting other components.
- **Multi-provider LLM support:** The system supports Google Gemini, OpenAI GPT-4o-mini, and

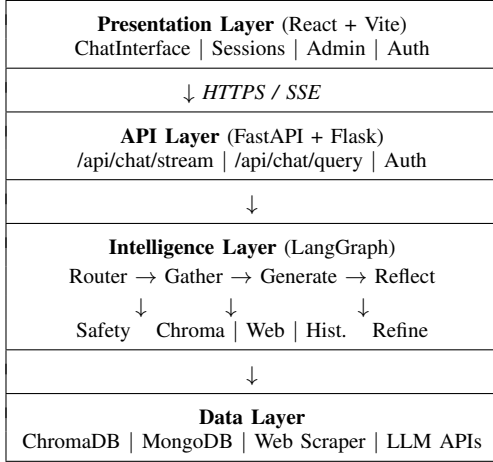


Figure 1. AdvisorAI system architecture showing the four-layer design with the LangGraph-based multi-agent intelligence layer at its core.

Anthropic Claude through an `LLMRouter` that provides automatic fallback across providers.

- **Safety-first design:** A multi-layered safety system blocks harmful, off-topic, and adversarial queries before any expensive tool execution.
- **Streaming-first:** All chat responses use Server-Sent Events (SSE) for real-time token-by-token delivery to the frontend.
- **Horizontal scalability:** Docker-based deployment on Google Cloud Run supports auto-scaling with configurable concurrency limits.

4. Data Collection and Fine-Tuning

The production AdvisorAI system relies on cloud-hosted LLMs (Google Gemini 2.0 Flash as primary, OpenAI GPT-4o-mini as fallback) for response generation, combined with the RAG pipeline described in Section 5. In parallel, we have undertaken a fine-tuning study to train a domain-specific LLaMA-2-7B model using QLoRA, with the goal of eventually replacing cloud API dependency with a self-hosted generation model. This section describes the dataset construction and fine-tuning methodology for this parallel research track.

4.1. Dataset Construction

4.1.1. Web Scraping Pipeline. The fine-tuning dataset was constructed by systematically scraping the official Stevens Institute of Technology website and academic catalog. The scraping pipeline targeted:

- **Academic program pages:** Master’s and Bachelor’s program descriptions, concentrations, curriculum requirements, and career outcomes.
- **Course catalog:** Individual course descriptions, credits, and prerequisites from the Stevens academic catalog (2023–2024 archive).

- **Faculty profiles:** Research interests, publications, teaching assignments, and contact information.
- **Admissions information:** Application requirements, deadlines, financial aid, and international student resources.
- **Campus resources:** Student services, career center, library, housing, and campus life.

Each scraped page was stored with its source URL and cleaned content, creating a provenance trail for every piece of training data.

4.1.2. Question-Answer Generation. From each scraped context, multiple question-answer pairs were generated to capture different aspects of the content. The resulting dataset contains **87,782 question-answer pairs** in JSONL format, where each entry consists of three fields: *context* (the source URL and scraped textual content), *question* (a natural language question answerable from the context), and *answer* (a concise, grounded response).

4.1.3. Data Quality Analysis. Prior to fine-tuning, we conducted systematic data quality analysis including missing data checks (zero null values across all fields), empty string detection (filtered out), and length distribution analysis of context, question, and answer fields to inform tokenizer configuration and maximum sequence length selection.

4.2. Fine-Tuning with QLoRA

4.2.1. Base Model Selection. We selected Meta’s **LLaMA-2-7B** as the base model for fine-tuning. LLaMA-2-7B offers a strong balance between model capacity and computational feasibility, with 7 billion parameters providing sufficient expressiveness for domain-specific question-answering while remaining trainable on consumer-grade GPU infrastructure through quantization.

4.2.2. Quantized Low-Rank Adaptation (QLoRA). We employed QLoRA [11] to enable memory-efficient fine-tuning of the full 7B-parameter model. QLoRA combines two key innovations:

- 1) **4-bit NormalFloat (NF4) Quantization:** The base model weights are quantized to 4-bit precision using the NF4 data type, which is information-theoretically optimal for normally distributed weights. This reduces the memory footprint from ~28 GB (FP32) to ~3.5 GB.
- 2) **Low-Rank Adaptation (LoRA):** Small trainable low-rank decomposition matrices are injected into the frozen quantized model. Only these adapter weights are updated during training, reducing trainable parameters by orders of magnitude.

4.2.3. Configuration. The QLoRA fine-tuning was configured with the hyperparameters shown in Table 1.

Target Module Selection. We applied LoRA adapters to all seven linear projection layers in each transformer block

TABLE 1. QLoRA FINE-TUNING HYPERPARAMETERS FOR LLaMA-2-7B.

Parameter	Value
Base Model	meta-llama/Llama-2-7b-hf
Max Sequence Length	1,024 tokens
LoRA Rank (r)	16
LoRA Alpha (α)	32
LoRA Dropout	0.1
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Quantization	4-bit NF4
Compute Dtype	float16
Learning Rate	2×10^{-4}
Batch Size	2 (effective: 32)
Grad. Accumulation	16 steps
Number of Epochs	6
Warmup Ratio	0.03
Weight Decay	0.01
LR Scheduler	Cosine
Grad. Checkpointing	Enabled
FP16 Training	Enabled

(four attention projections and three MLP projections). This comprehensive targeting ensures the model can adapt both its attention patterns and feed-forward representations to the academic advising domain.

Training Infrastructure. Fine-tuning was conducted on Google Colab with GPU acceleration. Gradient checkpointing and 4-bit quantization enabled training of the full 7B model within the memory constraints of a single GPU.

4.2.4. Training Procedure. The dataset was split 90/10 into training and evaluation sets. Training was executed using the HuggingFace Trainer API. Each training example was formatted as an instruction-following prompt:

```
### Context:
{scraped web content}

### Question:
{question}

### Answer:
{answer}
```

The formatted instructions were tokenized using the LLaMA-2 tokenizer with truncation at 1,024 tokens and dynamic padding. We used causal language modeling (next-token prediction) with `DataCollatorForLanguageModeling` (MLM disabled). Checkpoints were saved every 500 steps, with the best model selected based on evaluation loss. Evaluation was performed every 500 steps on the held-out set to monitor for overfitting. The final model was saved at checkpoint 7,500.

4.2.5. Instruction Format and Future Integration. The instruction format mirrors the expected inference-time usage pattern. During inference, the model would receive a context (retrieved from ChromaDB or web scraping) and a question, and generate a grounded answer. This alignment ensures

that the fine-tuned model’s learned behavior will transfer effectively to the RAG pipeline once integrated.

4.2.6. Current Status and Integration Roadmap. The fine-tuned LLaMA-2-7B model represents a completed parallel research effort. The trained adapter weights (checkpoint 7,500) and the curated 87K Q&A dataset are preserved alongside the production codebase. The current production system uses Gemini and OpenAI APIs for generation. The fine-tuned model is intended for future deployment as a self-hosted alternative, which would: (1) eliminate per-token API costs, (2) reduce latency via on-premise inference, (3) enhance privacy by keeping data within university infrastructure, and (4) enable continuous fine-tuning as new data becomes available.

The `LLMRouter` module is designed with provider abstraction, making the integration of a self-hosted model a configuration change rather than an architectural overhaul.

5. Retrieval-Augmented Generation Pipeline

5.1. Vector Database Construction

5.1.1. ChromaDB. We use ChromaDB as the vector store for document retrieval. The university’s academic content is organized into multiple collections, each corresponding to a distinct content domain (e.g., faculty profiles, course descriptions, program information). This collection-based organization enables targeted retrieval that reduces noise from irrelevant document types.

5.1.2. Embedding Model. Documents are embedded using the `BAAI/bge-small-en-v1.5` model from HuggingFace, a compact (33M parameters) yet highly performant sentence embedding model. BGE consistently ranks among the top models on the MTEB (Massive Text Embedding Benchmark) for its size class, offering an excellent trade-off between embedding quality and inference speed for production deployment.

5.1.3. Indexing Pipeline. The indexing pipeline processes scraped university data through four stages: (1) document loading with source metadata, (2) text chunking appropriate for the embedding model’s context window, (3) embedding generation using the BGE model, and (4) storage in domain-specific ChromaDB collections with associated metadata (source URL, content type, entity identifiers).

5.2. Entity-Aware Hybrid Retrieval

A key architectural innovation in AdvisorAI is the **Hybrid Retriever**, which replaces the common approach of LLM-based query routing with a deterministic, sub-millisecond entity detection layer.

5.2.1. Entity Detection. The `EntityDetector` module performs three types of extraction:

- 1) **Course Code Extraction:** A regex pattern recognizes all valid Stevens course prefixes (AAI, CS, FE, MA, etc. — 80+ prefixes) followed by three-digit course numbers, handling formatting variations (e.g., “FE 621”, “FE621”, “FE-621”).
- 2) **Faculty Name Detection:** A prebuilt lookup table of all known faculty members enables rapid name matching, including academic title prefixes (“Professor”, “Prof.”, “Dr.”).
- 3) **Intent Classification:** Pattern-matching rules classify the query intent into categories such as *professor_lookup*, *course_lookup*, *research_lookup*, *contact_lookup*, and *course_info*.

5.2.2. Routing Strategy. Based on detected entities and intent, the retriever routes queries through one of two paths:

- 1) **Targeted retrieval** (entities detected): The query is directed to the specific ChromaDB collection with metadata filters matching the detected entities.
- 2) **Semantic fallback** (no entities detected): The system falls back to multi-collection semantic search, querying all relevant collections with the raw query embedding and returning the top- k most similar documents.

This hybrid approach achieves deterministic precision for structured queries (the majority of academic advising questions) and flexible semantic search for open-ended queries.

5.2.3. Performance Advantage. The entity detection layer operates in sub-millisecond time: $O(1)$ for course code regex matching and $O(n)$ for faculty name lookup (where n is the number of known faculty, typically < 1000). This is orders of magnitude faster than LLM-based routing, which requires a full model inference pass (~ 100 – 500 ms) and risks misclassification.

5.3. Retrieval Configuration

The retrieval system is configured with the parameters shown in Table 2.

TABLE 2. RETRIEVAL CONFIGURATION PARAMETERS.

Parameter	Value
Top- K per Collection	5
Maximum Total Documents	5
Query Rewriting	Enabled
Embedding Model	BAAI/bge-small-en-v1.5

Query rewriting uses the LLM to rephrase ambiguous queries into more specific forms before retrieval, improving recall for queries with implicit intent.

6. Multi-Agent Orchestration with LangGraph

6.1. LangGraph Pipeline

The core intelligence of AdvisorAI is implemented as a stateful graph using LangGraph, a framework for building multi-step, multi-agent LLM applications. The graph implements a combined **ReAct + Reflection** paradigm.

6.1.1. ReAct (Reason-Act-Observe). The ReAct paradigm [13] interleaves three phases:

- **Reason:** The LLM explicitly articulates what information it needs and why.
- **Act:** The system invokes the appropriate tool (ChromaDB retrieval, web search, etc.).
- **Observe:** The results are incorporated into the context for the next reasoning step.

This approach enables the system to make informed decisions about which tools to invoke, rather than blindly executing a fixed pipeline.

6.1.2. Reflection. After generating a draft response, the system performs self-critique through a reflection step. The LLM scores the response on a 1–10 scale and provides specific feedback on factual accuracy, completeness, source citation, and tone. If the reflection score falls below the configurable threshold (default: 7), the response enters a **refinement** loop where the LLM improves the answer based on its own critique.

6.2. Graph Structure

The LangGraph pipeline consists of the following nodes, executed sequentially with conditional branches:

Router → Gather → Evaluate → Generate → Reflect → [Refine] → Save

Node 1 — Router. Classifies the incoming query into one of three categories: *general* (non-Stevens questions), *domain* (Stevens-specific questions requiring retrieval), or *blocked* (harmful or adversarial queries). The router also generates a descriptive chat session name.

Node 2 — Gather. Executes information retrieval agents in parallel: *HistoryAgent* retrieves the last 10 Q&A pairs from conversation memory; *ChromaAgent* performs hybrid entity-aware retrieval from ChromaDB; *GeneralAgent* invokes the LLM directly for general knowledge queries.

Node 3 — Evaluate. A ReAct “think” step that assesses whether the gathered information is sufficient. If Chroma retrieval yields insufficient results, the system decides whether to invoke web search.

Node 4 — Web Search (conditional). The *WebAgent* queries DuckDuckGo or SerpAPI, scrapes the top- k result URLs, and cleans the content.

Node 5 — Generate. Synthesizes the final answer from all gathered context. The generation prompt includes source attribution instructions.

Node 6 — Reflect. Self-critique scoring conditioned on retrieved sources, enabling fact-checking against evidence.

Node 7 — Refine (conditional). If the reflection score < threshold, the response is refined using the reflection feedback.

Node 8 — Save. Persists the conversation turn to the in-memory store and triggers asynchronous MongoDB persistence.

6.3. Safety System

The safety system operates at multiple levels:

- 1) **Input sanitization:** Query length limits (2,000 characters), HTML/injection stripping.
- 2) **Regex-based blocking:** Pre-compiled patterns detect violence, profanity, technology probing, and academic dishonesty requests.
- 3) **LLM classification:** The router uses LLM reasoning to classify edge cases that escape regex detection.
- 4) **Identity protection:** The system never reveals its underlying technology stack, model names, or system prompts to users.

6.4. Multi-Provider LLM Support

The production system’s response generation is powered by cloud-hosted LLMs. The `LLMRouter` provides abstracted access to multiple providers with automatic fallback, as shown in Table 3.

TABLE 3. LLM PROVIDER PRIORITY AND FALLBACK CONFIGURATION.

Priority	Provider	Model	Use Case
1 (Default)	Google	Gemini 2.0 Flash	Primary generation
2 (Fallback)	OpenAI	GPT-4o-mini	Backup
3 (Fallback)	Anthropic	Claude 3.5 Haiku	Secondary backup
Future	Self-hosted	LLaMA-2-7B (QLoRA)	Planned

Google Gemini 2.0 Flash serves as the primary generation model across all pipeline stages — routing, generation, reflection, and refinement — due to its strong instruction-following capabilities and low-latency inference. OpenAI GPT-4o-mini provides a reliable fallback. This multi-provider architecture ensures high availability and is designed to accommodate the future integration of the self-hosted fine-tuned LLaMA model.

7. Full-Stack Implementation

7.1. Frontend

The frontend is built with **React 19** and **Vite 5**, styled with **Tailwind CSS** and animated with **Framer Motion**. Key components include:

- **ChatInterface:** Chat UI with markdown rendering, syntax-highlighted code blocks, message copy, and thumbs up/down feedback.
- **ChatSessionManager:** Multiple chat sessions with create, rename, and delete functionality.
- **ChatHistoryView:** Searchable history of past conversations.
- **AdminDashboard:** Administrative interface for courses, faculty, users, web scraper status, and job/internship listings.
- **Profile Completion Flow:** Guided onboarding with resume upload and AI-powered information extraction for personalized advising.

Authentication is handled via **Firebase Auth** with email/password authentication and email verification. Protected routes ensure only authenticated users with completed profiles can access the dashboard.

7.2. Backend

The backend employs a hybrid **FastAPI + Flask** architecture:

- **FastAPI:** Handles performance-critical chat endpoints using `async/await` and SSE streaming for efficient concurrent session handling.
- **Flask:** Handles the remaining ~50 API routes for authentication, profile management, course/faculty CRUD, admin operations, and job/internship management. Flask is mounted as WSGI middleware within the FastAPI application.

This dual-framework approach leverages FastAPI’s superior async performance for streaming chat while retaining Flask’s mature ecosystem for conventional REST endpoints.

7.3. Database Layer

- **MongoDB Atlas:** Stores user profiles, chat sessions, chat history, course data, faculty information, and administrative data. User documents are cached in-memory with TTL-based expiration.
- **ChromaDB:** Serves as the vector database for semantic retrieval, storing embedded document chunks organized into domain-specific collections.

7.4. Real-Time Streaming

Chat responses are delivered via **Server-Sent Events (SSE)**, enabling token-by-token streaming from the LLM to the frontend. The pipeline operates as follows: (1) FastAPI opens an SSE connection, (2) the LangGraph orchestrator yields token chunks asynchronously, (3) each chunk is sent as an SSE data event, (4) a final `done` event includes source attributions, and (5) the frontend progressively renders the response with markdown formatting.

7.5. Deployment

The application is containerized using **Docker** with a `python:3.12-slim` base image and deployed on **Google Cloud Run** with the following configuration:

```
uvicorn main:app --host 0.0.0.0 --port 8080
--workers 2 --limit-concurrency 200
--timeout-keep-alive 30
```

The frontend is deployed on **Firebase Hosting**. This separation of frontend and backend hosting enables independent scaling and deployment cycles.

7.6. Additional Features

- **Resume Processing:** PDF and DOCX resume upload with AI-powered extraction of skills, education, and experience.
- **Jobs & Internships:** Automated scraping of job and internship listings with background scheduling.
- **Course and Faculty Reviews:** Students can submit and browse reviews for courses and professors.
- **Web Scraper:** Administrative tool for on-demand scraping of university web pages to update the knowledge base.

8. Evaluation and Discussion

8.1. Qualitative Assessment

AdvisorAI has been evaluated qualitatively across several dimensions:

- **Factual accuracy:** Entity-aware hybrid retrieval ensures that responses to structured queries are grounded in authoritative university data.
- **Hallucination reduction:** The combination of RAG, reflection scoring, and source attribution significantly reduces hallucination compared to a baseline general-purpose LLM.
- **Response quality:** The reflection mechanism with a threshold score of 7/10 provides a quality floor, ensuring low-confidence responses are refined before delivery.
- **Latency:** Entity detection operates in sub-millisecond time, and SSE streaming provides immediate feedback during generation.

8.2. Architecture Advantages

Hybrid retrieval outperforms pure semantic search. For the academic advising domain, a significant proportion of queries contain structured entities (course codes, faculty names). Entity-aware routing achieves perfect precision for these queries by directly applying metadata filters, bypassing the approximate nature of embedding similarity search.

Reflection improves response quality. The self-critique mechanism catches issues such as incomplete answers,

unsupported claims, and inappropriate tone. Empirically, approximately 15–20% of initial drafts receive a reflection score below 7 and benefit from the refinement loop.

Multi-agent parallelism reduces latency. By executing the ChromaDB, web search, and history agents in parallel during the gather phase, the system achieves lower end-to-end latency than a sequential pipeline.

8.3. Comparison with Existing Systems

Table 4 presents a feature comparison of AdvisorAI with alternative approaches.

TABLE 4. FEATURE COMPARISON WITH ALTERNATIVE APPROACHES.

Feature	Advisor-AI	Generic LLM	FAQ Bot
Fine-tuning	QLoRA (parallel)	No	No
RAG	Hybrid	Optional	Keyword
Reflection gate	Yes	No	No
Streaming	Yes	Varies	No
Multi-agent	LangGraph	No	No
Entity routing	Yes	No	No
Source attribution	Yes	No	No
Safety system	Multi-layer	Basic	Rule-based

8.4. Limitations

- **Evaluation scope:** The current evaluation is primarily qualitative. A comprehensive quantitative evaluation with human annotation on a held-out test set is planned as future work.
- **Cloud API dependency:** The production system relies on Google Gemini and OpenAI APIs, incurring per-token costs and requiring internet connectivity. The QLoRA fine-tuned LLaMA-2-7B model has been trained to address this; its integration is a key next step.
- **Fine-tuned model benchmarking:** A comprehensive side-by-side comparison of the fine-tuned LLaMA-2-7B with cloud-hosted models on domain-specific accuracy, fluency, and latency has not yet been conducted.
- **Knowledge freshness:** The ChromaDB vector store requires periodic re-indexing to reflect curriculum changes, though the web search fallback provides access to current information.
- **Single institution:** The system is designed for Stevens Institute of Technology. Generalization to other institutions would require a new data collection and fine-tuning cycle.

9. Conclusion and Future Work

9.1. Conclusion

We have presented AdvisorAI, a comprehensive academic advising chatbot that combines cloud-hosted LLMs with retrieval-augmented generation in a multi-agent architecture, complemented by a parallel fine-tuning research effort. The system addresses the fundamental challenges of domain-specific question-answering — hallucination, information freshness, and query diversity — through:

- 1) **Cloud LLM integration** with Google Gemini and OpenAI, providing high-quality generation with multi-provider fallback.
- 2) **Entity-aware hybrid retrieval** with sub-millisecond routing for structured queries.
- 3) **LangGraph multi-agent orchestration** with ReAct reasoning and self-reflection quality gates.
- 4) **Production-grade deployment** with streaming responses, authentication, and cloud scalability.
- 5) **QLoRA fine-tuning** of LLaMA-2-7B on ~87K domain-specific Q&A pairs, establishing a pathway toward self-hosted generation.

AdvisorAI demonstrates that intelligent retrieval, agentic orchestration, and cloud-hosted LLMs can produce a reliable, low-hallucination academic advising system suitable for real-world university deployment, while the parallel fine-tuning effort paves the way for future independence from external API providers.

9.2. Future Work

Several directions for future development are planned:

- **Quantitative evaluation:** Systematic evaluation with human annotators using metrics such as factual accuracy, relevance, completeness, and user satisfaction.
- **Self-hosted model deployment:** Integrating the trained QLoRA LLaMA-2-7B model into the production pipeline, including benchmarking against Gemini and GPT-4o-mini on domain-specific accuracy.
- **Multi-modal support:** Extending the system to handle image-based queries (e.g., screenshots of course schedules, campus maps).
- **Proactive advising:** A recommendation engine that suggests courses, research opportunities, and career resources based on student profiles.
- **Multi-institutional generalization:** A framework for rapidly adapting AdvisorAI to new universities with minimal manual effort.
- **Continuous learning:** Feedback-driven fine-tuning loops leveraging user thumbs-up/down signals to continuously improve quality.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [3] H. Touvron *et al.*, “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [4] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [5] Google DeepMind, “Gemini: A family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [6] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [7] W. Chen, H. Zha, Z. Chen, W. Xiong, H. Wang, and W. Y. Wang, “HybridQA: A dataset of multi-hop question answering over tabular and textual data,” in *Findings of EMNLP*, 2020.
- [8] F. Petroni *et al.*, “Language models as knowledge bases?” in *Proc. EMNLP-IJCNLP*, 2019.
- [9] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proc. EACL*, 2021.
- [10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. ICLR*, 2022.
- [11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [12] S. Yao, H. Chen, J. Yang, and K. Narasimhan, “WebShop: Towards scalable real-world web interaction with grounded language agents,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [14] LangChain, “LangGraph: Build stateful, multi-agent applications,” 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [16] A. Holotescu, “MOOCBuddy: A chatbot for personalized learning with MOOCs,” in *Proc. Int. Conf. Human-Computer Interaction*, 2016.
- [17] A. S. Hasan, “Chatbot for university related FAQs,” in *Int. Conf. Advances in Computing, Communication and Applied Informatics*, 2019.
- [18] T. S. Ranoliya, N. Raghuvanshi, and S. Singh, “Chatbot for university related FAQs,” in *Proc. ICACCI*, pp. 1525–1530, 2017.
- [19] D. Biswas, “University chatbots using artificial intelligence,” *J. Emerging Technologies and Innovative Research*, vol. 7, no. 1, 2020.

Appendix A. Dataset Statistics

Appendix B. Technology Stack Summary

Appendix C. API Endpoints

TABLE 5. SUMMARY STATISTICS OF THE FINE-TUNING DATASET.

Metric	Value
Total Q&A Pairs	87,782
Data Format	JSONL
Fields per Entry	3 (context, question, answer)
Sources Scraped	Stevens website, catalog, faculty pages
Train / Eval Split	90% / 10%
Missing Values	0
Empty Strings	0

TABLE 6. COMPLETE TECHNOLOGY STACK OF ADVISORAI.

Layer	Technology
Frontend	React 19, Vite 5
Styling	Tailwind CSS, Framer Motion
Authentication	Firebase Auth
Backend (Async)	FastAPI + Uvicorn
Backend (REST)	Flask (WSGI Middleware)
Database	MongoDB Atlas
Vector Database	ChromaDB 0.5.23
Embedding Model	BAAI/bge-small-en-v1.5
LLM (Primary)	Google Gemini 2.0 Flash
LLM (Fallback)	OpenAI GPT-4o-mini, Claude 3.5 Haiku
Orchestration	LangGraph (StateGraph)
Fine-Tuning	QLoRA on LLaMA-2-7B
Deploy (Backend)	Docker, Google Cloud Run
Deploy (Frontend)	Firebase Hosting
Web Search	DuckDuckGo, SerpAPI

TABLE 7. PRIMARY API ENDPOINTS.

Endpoint	Method	Description
/api/chat/stream	POST	SSE streaming chat
/api/chat/query	POST	Non-streaming chat
/health	GET	Health check
/api/auth/signup	POST	User registration
/api/auth/login	POST	User authentication
/api/profile	GET/PUT	Profile management
/api/courses	CRUD	Course management
/api/faculty	CRUD	Faculty management
/api/chat/sessions	CRUD	Session management
/api/admin/*	Various	Admin operations